MS2 Summary - Predicting Pedestrian Flow

Timo Schlumpf | 5 November 2026 | gaeja / P15

1. What I achieved in this milestone

Implemented a Python CLI for live public-data ingestion, timestamp-based historical backfill, versioned Parquet features, chronological evaluation, baseline prediction archiving and matured-label MAE monitoring. Shared feature definitions serve historical and live paths. Pinned dependencies, Docker, CI and an hourly GitHub Actions workflow are included. First files to inspect: README.md, src/gaeja/features.py, tests/, .github/workflows/features.yml.

Important correction: the export pads Lintheschergasse with zero placeholders before installation. Filtering verified measured records gives 23,184 real hourly labels from 2024-03-13 23:00 through 2026-11-04 22:00 UTC, not five years. Other sensors stay spatial context.

Availability: the public export was about 16 hours behind on the first fetch. Historical backfill therefore defaults to an explicit, unverified 24-hour publication delay; live features use observed first-seen times. At a decision 10 minutes before the next clock hour, lag1 is not yet complete. Persistence uses the latest available count with a 48-hour staleness cap, rather than pretending lag1 is available.

Evaluation: frozen split 2026-10-01 00:00 UTC; training labels must be published before the first holdout decision. Fixed model parameters; 839 identical test hours, through 2026-11-04 22:00 UTC. Historical availability remains an assumption, not a live accuracy claim.

Forecast	Holdout MAE
Latest available persistence	227.42
Seasonal naive (lag168)	77.45
Target lag/calendar model	64.37
Lag1 oracle (not deployable at cutoff)	84.16

Success threshold: MAE <= 69.71 (10% below the better deployable baseline). Initial model improvement: 16.9%. GTFS ablation: not yet possible on real point-in-time history.

Verified locally: 18 tests pass with warnings treated as errors; Ruff passes; live Hystreet ingestion, 48 Open-Meteo forecast rows, backfill, evaluation and a versioned baseline forecast ran. No live labels have matured yet. Data/reports stay gitignored; commands reproduce them.

Not claimed complete: official GTFS catalog returns HTTP 403; historical coverage is unsecured. Docker build and container CLI smoke test now pass. GitHub workflows must be pushed and observed before calling scheduling operational. Artifacts have 30-day retention. No school-holiday dates or weather-prototype provenance were invented.

2. Response to MS1 reviews

Every distinct editorial/reviewer concern is addressed below; shared labels are combined. References are repository files, not claims of uncreated commits.

E1 - Partly. Named the official Swiss GTFS catalog/cookbook in README.md. gtfs-discover and gtfs-import provide resource discovery, hashed provenance and availability-aware import (src/gaeja/gtfs.py). Catalog access returns 403 here; earliest archive fallback now finds 2024/2025/2026 snapshots and downloads a May 2026 ZIP. Original publication times remain unverified. A user-downloaded 2026-11-04 feed was imported at 2026-11-05 15:34 UTC; 113 next-hour HB arrivals verified. No historical backfill gain claimed.

A1 / B5 - Changed. src/gaeja/evaluation.py compares persistence, weekly baseline and a target-lag/calendar model on the same holdout. Success is 10% below the better deployable baseline. Lag1 is separately shown as an unavailable oracle; paired GTFS ablation is implemented but awaits real schedule history. Results and cutoff rationale are on page 1.

B2 - Partly. Static schedules may only duplicate weekly calendar structure. The paired with/without-GTFS experiment is implemented on identical covered rows, with no real result claimed yet. Keep this as the MS3 headline, including a null result; consider delays/occupancy only if justified (README.md, evaluation.py).

E2 - Partly. Replaced per-event scope with next-hour Zurich HB arrival counts. Importer handles service exceptions, overnight times and DST; synthetic tests exercise these rules. The user-supplied national feed now imports and yields real hourly arrivals (src/gaeja/gtfs.py, tests/test_pipeline.py).

A3 / B6 - Partly. Named Open-Meteo; ingest archives five next-hour forecast variables with retrieval-time availability. Historical stitched forecasts are not used as publication vintages. Verified forecast backfill and a separate observation source remain outstanding (sources.py, features.py, README.md).

A2 - Partly. First-seen timestamps and recent latency logs are implemented; first fetch found the export around 16 hours stale. Live persistence selects the actual latest available interval, and short lags stay missing. Historical default is a labeled 24-hour delay assumption, not validated historical latency. More scheduled samples are needed (sources.py, features.py, data/latency.parquet after ingestion).

B3 - Changed. Core monitor scores immutable archived forecasts against matured labels and both baselines, logging MAE, sample counts and paired-row comparisons. No-matured-label state is explicit; no zero-error claim (evaluation.py).

B4 - Changed. README.md FTI diagram connects the feature store to inference and shows future-known GTFS/weather entering backfill and inference. Model registry is clearly marked MS3/MS4 work, not a completed component.

E3 - Not yet. README.md explicitly says no weather prototype existed in this repository and its provenance was not provided. Current ingestion is new code; original prototype provenance must still be supplied by the author.

E4 - Changed. Sensor 670 remains the only label; other sensors are lagged spatial features, never pooled targets. The comparison model is target-lags/calendar only (sources.py, features.py, evaluation.py).

B1 - Rejected. The target remains counted passers-by, explicitly not customers. README.md narrows the one-hour business rationale to immediate preparation/queue management/promotions; customer conversion calibration remains a future extension.

Submission checklist: commit source and this PDF, push, observe CI and the scheduled pipeline, then tag/push ms2. Resolve GTFS history access and verify publication cadence. No commit, push, milestone tag or cloud deployment is claimed by this local implementation.